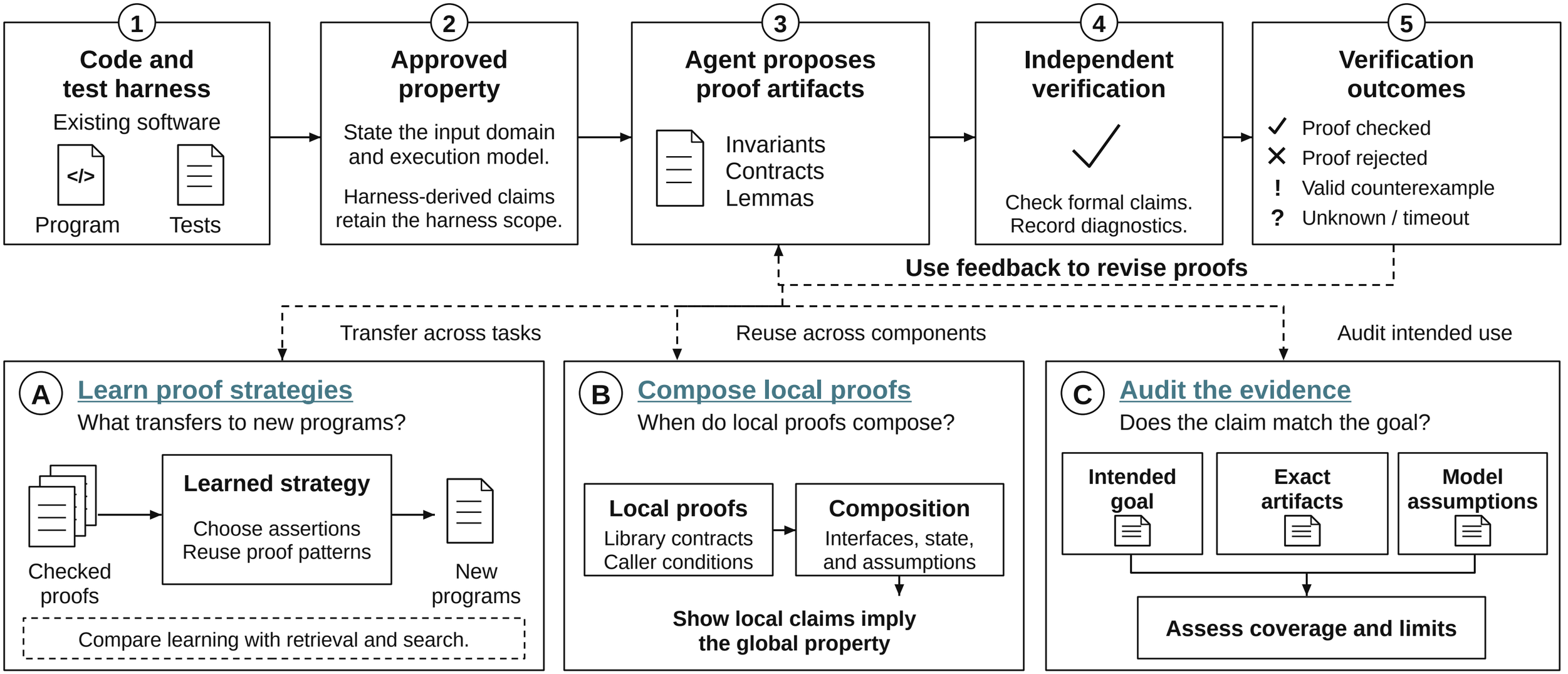
Learning Reusable Formal Reasoning

Proposed research

Learn from verification, compose proofs, and assess what the evidence supports



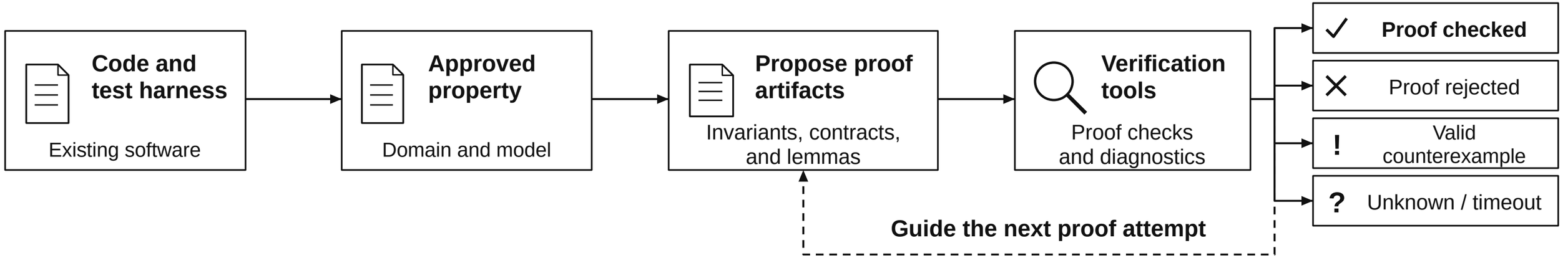
Goal: reusable, independently checked software evidence

Every guarantee is limited by the approved property, model, assumptions, and trusted checking.

Learning Reusable Proof Strategies

Proposed research

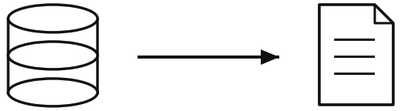
What transfers from training to unfamiliar programs?



Compare three mechanisms under matched data, tools, and total cost

Retrieve

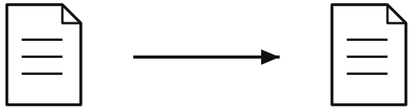
Reuse a stored proof artifact.



Recheck it for the new obligation.

Refine

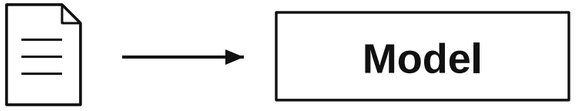
Revise within one task.



Use feedback with model weights fixed.

Learn

Learn across training tasks.



Compare proofs, checked steps, and revisions.

Held-out evaluation

Freeze model weights and the retrieval bank before testing.

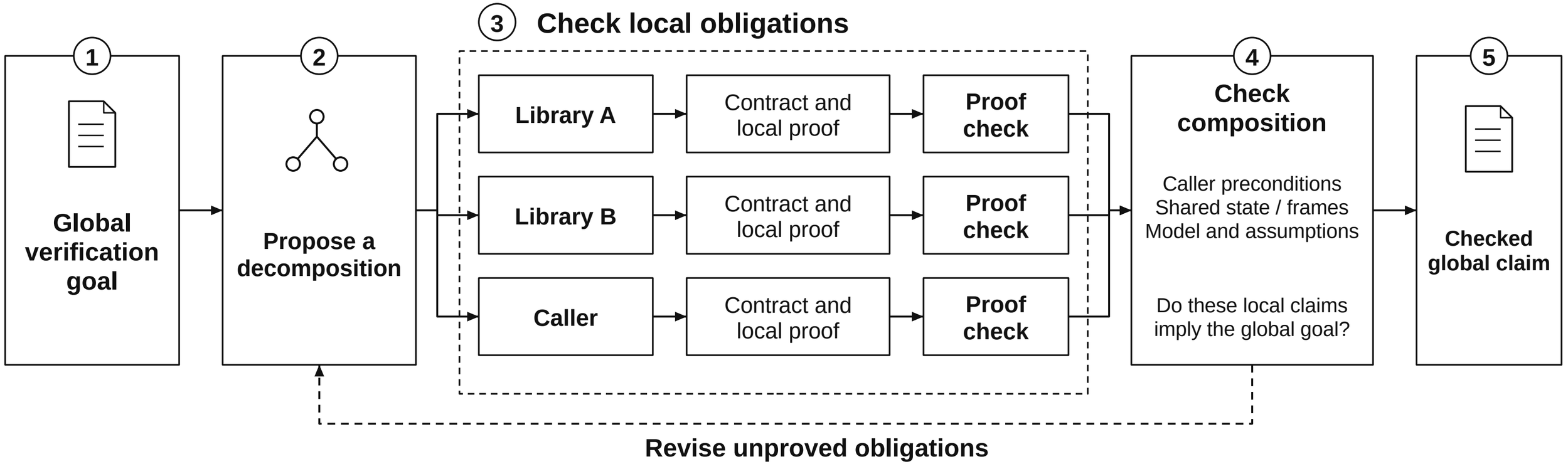
Keep related programs in the same split. Report retrieval, refinement, and learned transfer separately.

Proof rejection does not refute a claim. Validate counterexamples against the stated claim and model.
A failed auxiliary invariant need not imply a software bug. Unknown results and timeouts are inconclusive.

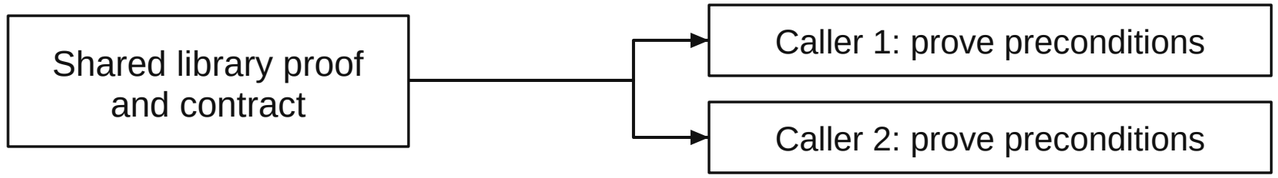
Compositional Verification

Proposed research

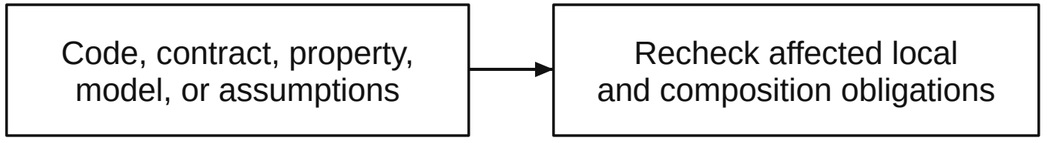
Check local proofs and the argument that connects them



Reuse requires each caller to meet the library contract



After any relevant change

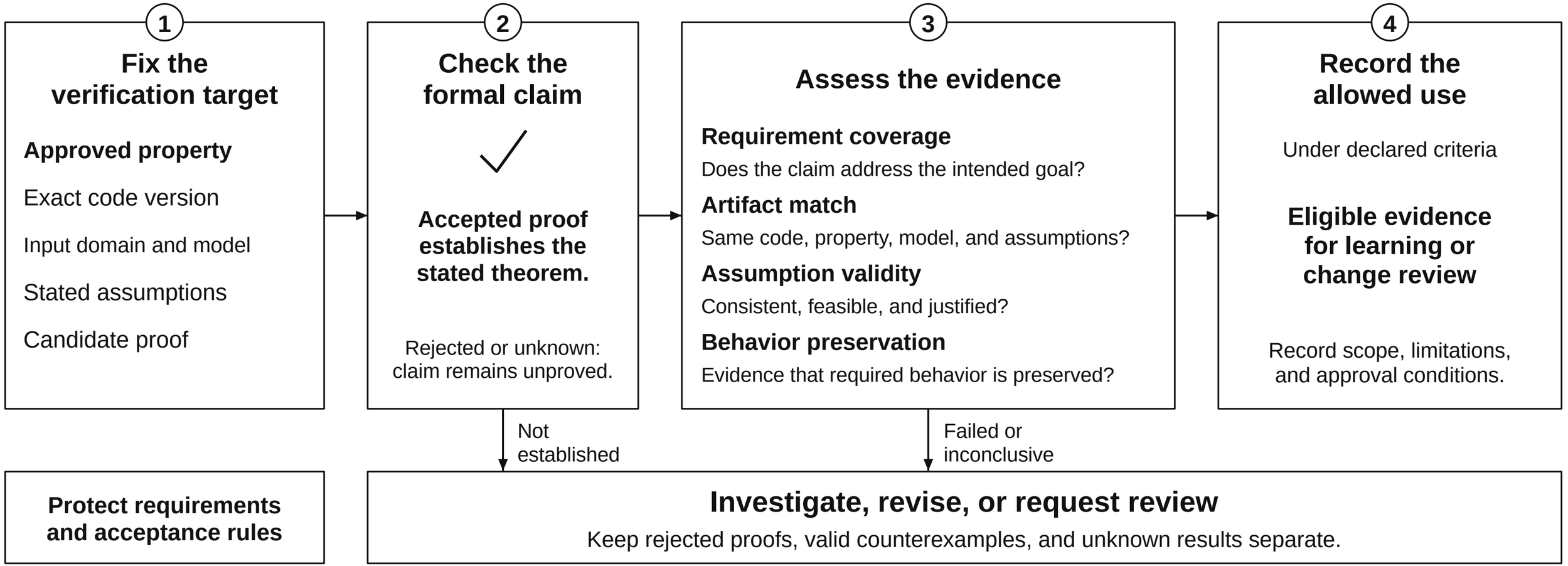


Composition requires discharged caller obligations and a checked argument from local claims to the global property.

Trustworthy Verification Feedback

Proposed research

A checked theorem still needs a review of its scope and intended use



Failure cases to audit

Weak property
Important requirements are left unspecified.

Unrealistic assumptions
Conditions are inconsistent or never feasible.

Behavior removed
The patch disables required functionality.

A checked proof establishes its theorem. Requirement tests can still miss required behavior.